

The Walk Across the Last Mile: a Kernel-Guarded Search That Invents Its Own Waypoints

Karl Olivet*

Preprint — August 2026

Abstract

A companion paper argued that in mechanised proof search the last mile is located, not crossed: a search that fails should name exactly what it lacks. This paper reports the next step — a *walker* that crosses the stretch the previous system could only locate. Given a goal that backward chaining alone cannot close — measured: on a four-element reassociation over a derived operation $a \oplus b := (a + b) + 1$, chaining exhausts its budget in 692 s — the walker mines the goal’s own subterms for repeated patterns, rediscovers $\oplus$ without being told it exists, conjectures laws for it from an open list of schemas, lets a numeric oracle kill the false ones in milliseconds, has the kernel certify the true ones, and re-attempts the goal on the *compressed* pool of derived lemmas alone. The same goal then closes, kernel-checked, end to end in 414 s — less time than chaining needs to fail. Two measured facts frame the contribution. First, compression is a *replacement*, not an addition: re-attempting on the derived lemma alone takes 72.6 s, while adding it to the raw laws takes 962 s — a factor of 13, because new knowledge enlarges the search space unless the old knowledge is set aside. Second, the walker’s own first component failed in a way worth reporting: ranking candidate patterns by size selected only fragments of the huge τ -developments (a triple- $\oplus$ term has $\approx 13,700$ nodes) and missed $\oplus$ entirely; the fix — pairing subterms by root signature — is itself a measured design law. As everywhere in this series, the kernel guards every step: a bad proposal costs a dead route, never a false theorem.

1 Introduction

The companion paper on failure reporting ended on a gate it refused to cross: its editorial plan states that no walker should be published before it has closed at

*Independent researcher. Contact: karl.olivet@gmail.com. Code and certified corpus: <https://github.com/Karl-0/bourbaki-theorie-des-ensembles>; every figure reported here was measured on 21 August 2026 and is recorded, with its experiment, in `article/marcheur/MESURES.md`.

least one goal that chaining alone cannot close. This paper begins by crossing that gate, measured on both sides, and everything else in it is a consequence of how the crossing was done.

The claim, in one paragraph. A goal that defeats a bounded backward search does not usually defeat it by depth alone. What is missing is a *waypoint*: an intermediate statement that, once certified and placed in the pool, makes the remaining distance short. The walker’s job is to invent candidate waypoints from the goal’s own structure, discard the false ones cheaply, pay the kernel’s price for the true ones, and then attempt the goal from the waypoint rather than from the raw laws. None of this requires the machine to be right: every candidate is judged — first by a numeric oracle that can only refute, then by the kernel that alone can certify. A wrong proposal costs seconds; an unsound one is impossible.

Contributions. All figures were measured on 21 August 2026 and are reproducible from the repository (Section 9).

- **The gate, crossed and asserted on both sides** (Sections 3 and 7): chaining alone fails on B_4 in 692 s; one unattended call to the walker closes it, kernel-checked, in 414 s. A test in the repository asserts both sides.
- **Replacement, not addition, measured three times** (Sections 3 and 7): the derived lemma alone, 72.6 s; with one more true lemma, 361 s; added to the raw laws, 962 s; all six certified lemmas at once, over 580 s and killed. Hence the walker’s compression ladder.
- **A miner that rediscovers the operation from the goal** (Section 5) — including the measured failure of its first version, which ranked by size and saw only τ -development noise.
- **A refutation oracle with a stated blind spot** (Section 6): one added layer makes derived-operation conjectures visible; what stays invisible cost a measured 11 s of failed certifications on this bench.

2 Background: the bench, the budget, the gap

We reuse the toy-algebra bench on which the failure-reporting paper measured its rewriting organs. The base theory provides cardinal addition $a + b$ and its two raw laws — iterated associativity and commutativity — as the only facts in the pool. On top of it, a derived operation is defined and never named anywhere in the repository:

$$a \oplus b := (a + b) + 1.$$

The engine’s budget is not a guess: the minimal raw rewriting chain for the three-element associativity of $\oplus$ was measured at five steps, and five is therefore the engine’s cap (`max_pas=5`); with a cap of three, that same goal failed. All experiments below inherit this measured budget.

The goal that crosses the gate is the four-element reassociation

$$B_4 : ((a \oplus b) \oplus c) \oplus d = a \oplus (b \oplus (c \oplus d)),$$

true, and out of reach of a five-step raw chain: each application of $\oplus$ -associativity costs about five raw steps, and B_4 needs two.

3 The gate, measured on both sides

experiment	verdict	time
chaining alone on B_4 (raw pool)	fails , one gap named	692.5 s
certifying the lemma $\oplus$ -assoc	closed, kernel-checked	4–7 s
B_4 on the <i>compressed</i> pool (lemma alone)	closed , 0 hypotheses	72.6 s
B_4 on the cumulative pool (raw + lemma)	closed	962.4 s
false variant F_4 on the compressed pool	stays open (correct)	79.1 s

Three readings, in decreasing order of importance.

The gate holds. The same goal, from the same pool, under the same budgets: chaining exhausts itself in 692s and returns a named gap; the two-step walk — certify the waypoint, then re-attempt from it — closes it. “Cannot close” here is not an asymptotic claim; it is the measured behaviour of the full 21-organ engine at its measured budget.

Compression is a replacement, not an addition. The most useful number in the table is the least intuitive one: adding the certified lemma to the raw laws makes the search *thirteen times slower* than replacing them with it (962s against 72.6s). Every fact in the pool is a rewriting candidate at every node; new knowledge enlarges the space unless the old knowledge is set aside. The walker therefore re-attempts goals on the derived lemmas *alone* — and says so in its journal, rather than silently.

Certification cost is noisy. The same lemma certified twice took 3.8 s and then 7.4s depending on cache state. We quote “a few seconds” and refuse a single figure.

4 The walker

The walker’s state is a pair (goal, pool of certified facts). One step:

1. **Sense.** Run the need organ. Closed: done. Open: a list of named gaps — the failure object of the companion paper.
2. **Mine.** Extract repeated term patterns from the goal itself, by anti-unification with shared slots, ranked by the MDL gain $(\text{occurrences} - 1) \times \text{size}$ — the compression criterion of the notion-mining organ, transposed from formulas to terms.
3. **Conjecture.** Instantiate law schemas on each binary pattern: commutativity, associativity, idempotence. The schema list is *open*; nothing proves these three suffice, and the paper does not claim it.
4. **Refute cheaply.** A numeric oracle looks for small counterexamples. A false conjecture dies in milliseconds, never in minutes of kernel time. The oracle can only refute: no counterexample authorises spending, proves nothing.
5. **Certify, compress, retry.** Surviving conjectures go to the need organ over the raw pool; the kernel judges. Certified lemmas form the compressed pool on which the goal is re-attempted. No progress in a round: stop, and return the terminal gaps — the walker, like the organ it wraps, fails by naming.

The safety principle is unchanged since the first proposer organ: the walker *suggests*, the kernel *judges*. No theorem object is built in the walker’s code; everything that enters the pool came out of the need organ, hence out of the kernel.

5 The miner, including its first failure

The miner answers the question the reader should be asking: *who told the machine about $\oplus$* ? Nobody. The pattern is recovered from the goal.

Version 1 failed, and the failure is a design law. The first miner collected the goal’s subterms and anti-unified the 24 largest. It found nothing usable: in this kernel, terms are opaque τ -developments, and a triple- $\oplus$ term has $\approx 13,700$ nodes, so the largest subterms are all *fragments of the developments’ insides* — and the small instances $a \oplus b$, the ones that carry the pattern, never entered the top-24. The top pattern was a three-slot context of MDL gain 13,737, useless for conjecturing, and $\oplus$ was absent from the list. Ranking by size selects noise.

Version 2 pairs by root signature. Subterms are grouped by root signature (tag, name, binder, arity) and each is anti-unified with its size-neighbours *within its group*: $O(n)$ pairs instead of $O(n^2)$, and two instances of the same operation necessarily meet. Occurrences are then counted over *all* subterms, and candidates are validated by reconstruction: a pattern applied to the pieces of its own generator must rebuild it identically — assemblages are hashable, so this is one $O(1)$ equality, never a structural guess. On B_4 the head pattern comes out with 6 occurrences and gain 4,045, and applying it to (a, b) yields exactly $\oplus(a, b)$: mining took 1.1 s.

6 The oracle had a hole, and one layer was enough

The refutation step exposed a measured blind spot. The oracle evaluates terms by a reconstruction table — numerals, their sums, their products — because terms here are opaque: there is nothing to descend into. But $\text{succ}(\text{sum})$ was in no entry, so *every* conjecture about a derived operation evaluated to “unknown”: invisible, neither refutable nor authorised.

One layer fixed it: for every term already in the table with value v , add its successor with value $v + 1$. After the extension, the idempotence schema for $\oplus$ — false — dies in under a millisecond, before any kernel call; commutativity and associativity survive the sweep (about a second each) and proceed to certification. The limit is stated where the code is: one layer only. A nested derived term such as $\text{succ}(\text{succ}(a + b) + c)$ is still outside the table, the full closure would cost $|\text{table}|^2$, and on such conjectures the oracle stays silent — it never affirms.

7 The walk, end to end

Everything above described organs; this section reports the machine running them unattended. One call — `marcher( $B_4$ , raw pool)` — with a timestamped probe on its journal:

phase	window	duration
mining (4 patterns, gains 4045/1771/1155/895)	0–4.0 s	4.0 s
12 conjectures: 6 certified, 2 refuted, 4 failed	4.0–53.2 s	49.2 s
$\oplus$ -commutativity certified		20.2 s
$\oplus$ -associativity certified		16.8 s
idempotence refuted (at $x = 0$, resp. $x = 1$)		<0.1 s
retry, ladder rung 1 (the head pattern’s 2 lemmas)	53.2–414.2 s	361.0 s
closed — kernel-checked, 0 hypotheses, 22 axioms		414.2 s

The walk closes in 414 s a goal that chaining spends 692 s *failing* to close. Figure 1 puts the four searches of this paper side by side. But the number worth the section is how the retry got its shape.



Figure 1: Four searches on the same goal B_4 , same budgets, one machine (script: `scripts/gen_chronologie.py`, figures from `MESURES.md`). Chaining alone exhausts its budget and fails; the walk closes, and most of its time is the final retry — on the head pattern’s two lemmas only. The figure proves no generality: one goal, one bench, and failure durations depend on the measured budget caps.

The walk first failed by its own richness. The first version re-attempted the goal on *all* certified lemmas at once — six of them, since the miner’s lesser patterns also yield certifiable laws. That retry exceeded 580 s and was killed: the walker had grown its pool into its own obstacle, exactly the failing-by-growing signature the companion paper documents for raw search. The fix is the *compression ladder*: re-attempt on the head pattern’s lemmas alone, widen to the next pattern only on failure. Rung 1 sufficed. And a finer measurement sharpens the replacement law: the rung’s two lemmas ($\oplus$ -comm + $\oplus$ -assoc) close B_4 in 361 s, while $\oplus$ -assoc *alone* closes it in 72.6 s — even one true, on-topic, certified lemma too many costs a factor of five.

The price of the oracle’s blind spot, measured. Four conjectures (associativity and idempotence of the two lesser patterns) were out of the one-layer oracle’s fragment, went to certification, and failed there — about 11 s of the 414. That is what the stated limit of Section 6 costs on this bench: real, small, and on the journal rather than hidden.

8 What the walk does not find

9 Reproducibility

Every figure in this paper was measured on 21 August 2026 and recorded, with its experiment, in `article/marcheur/MESURES.md`; the experiment scripts are versioned beside it in `article/marcheur/scripts/` (`exp1_porte_directe.py`,

`exp3_pool_comprime.py`, `exp4_marche.py`). The gate itself is locked by tests in `outils_ia/decouvertes/autonomie/test_general.py`: a fast one asserting the miner rediscovers $\oplus$ and the oracle’s verdicts, and two slow ones asserting both sides of the gate and the false-variant guard. Durations are whole-call wall-clock on one machine (Python 3.13, Windows); the certification variance of Section 3 applies throughout.

10 Related work

Theory exploration. The closest relative of the mine–conjecture–refute–certify loop is theory exploration as practised by Hipster in Isabelle/HOL [JRSC14], in the QuickSpec lineage: enumerate candidate lemmas about given operations, filter them by testing, prove the survivors. Two differences matter here. Hipster explores *around a theory* — its input is a set of operations, its output a library; the walker is *goal-directed* — its input is one goal it must close, the operations are not given but mined from that goal’s own subterms, and a conjecture is only certified because a crossing needs it. And our setting is a Bourbaki τ -calculus on an LCF kernel, where terms are opaque developments: the mining step (and its measured failure) has no counterpart when terms are readable trees. The survey of automated conjecturing [ZT26] maps the wider landscape.

Counterexample before proof. Killing conjectures with small models before attempting proof is established Isabelle practice — Nitpick and its integration with Sledgehammer [BBN11]; recent work learns to generate formal counterexamples with language models [LLY⁺26]. Our oracle is deliberately humbler: a reconstruction table over numerals, extended by one measured layer, whose blind spots are stated rather than smoothed over — and whose verdicts are asymmetric by construction, refutation certain, silence otherwise.

Library learning and compression. The MDL criterion the miner ranks by — name what repeats, weighted by what naming saves — is the compression principle of wake-sleep library learning in DreamCoder [EWN⁺21]. What the bench adds is a caution those systems rarely measure: a learned abstraction only pays if it *replaces* the primitives in the search space; kept alongside them, it made our search thirteen times slower.

Learned proposers and guided search. GPT-*f* [PS20], HyperTree Proof Search [LLL⁺22] and ENIGMA [JCO⁺20] learn to propose or rank inference steps, with success measured over benchmark corpora. The walker shares their architecture-level premise — an unsound proposer behind an exact judge — but not their mechanism: it learns nothing, retains nothing between runs, and proposes from the structure of the one goal in front of it. The comparison it invites is

therefore not proof rates but failure behaviour: what remains when the run does not close.

Conjectures and refutations. The loop’s shape — propose a law, seek a counterexample, keep what survives — is Lakatos’s method reduced to its mechanical shadow; Pease’s computational model [Pea07] works the same ground from the philosophy side.

11 Limitations

One bench. The walker has closed one family of goals over one derived operation on one toy algebra. Nothing here claims generality: the paper’s claim is that the gate — close something chaining cannot — has been crossed with every step measured, and that two of those measurements (replacement-not-addition; size-ranking selects noise) are design laws that survive the bench. The schema list is open. The walker does not learn: it retains nothing between runs, and the title of the editorial plan for this paper — “learning to propose” — has deliberately *not* been kept, because the code does not yet earn it.

12 Conclusion

The companion paper’s thesis was that the last mile is located, not crossed: a search should fail by naming what it lacks. This paper kept the naming and added the crossing — and the crossing turned out to be made of the same material as the locating. The walker closes B_4 not by searching harder but by changing what it searches *with*: mine the goal for its own operation, refute the false laws for pennies, pay the kernel for the true ones, and then — the step the measurements insist on — *set the raw laws aside*. Every number in this paper that surprised us is the same law seen from a different side: knowledge that is not replaced is weight (72.6 s alone, 361 s with one extra lemma, 962 s cumulative, over 580 s with all six); and a walk that grows its pool without pruning fails by growing, exactly like the raw searches of the companion paper.

What the walker still is not: a learner. It retains nothing between runs; the learned-proposer organ that remembers which witnesses paid is wired to the need organ, not yet to the walk. Connecting them — so that the ladder’s first rung is ordered by remembered success rather than by MDL gain alone — is the natural next step, and this time the gate for claiming “learning” in a title is already written: the memory must change a measured outcome on a goal the memoryless walker fails.

References

- [BBN11] Jasmin C. Blanchette, Lukas Bulwahn, and Tobias Nipkow. Automatic proof and disproof in Isabelle/HOL. In *Proc. FroCoS*, 2011.
- [EWN⁺21] Kevin Ellis, Catherine Wong, Maxwell Nye, et al. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proc. PLDI*, 2021.
- [JCO⁺20] Jan Jakubův, Karel Chvalovský, Miroslav Olšák, Bartosz Piotrowski, Martin Suda, and Josef Urban. ENIGMA anonymous: Symbol-independent inference guiding machine. In *Proc. IJCAR*, 2020.
- [JRSC14] Moa Johansson, Dan Rosén, Nicholas Smallbone, and Koen Claessen. Hipster: Integrating theory exploration in a proof assistant. In *Proc. CISM*, 2014.
- [LLL⁺22] Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving. In *Proc. NeurIPS*, 2022.
- [LLY⁺26] Zenan Li, Zhaoyu Li, Kaiyu Yang, Xiaoxing Ma, and Zhendong Su. Learning to disprove: Formal counterexample generation with large language models. arXiv:2603.19514, 2026.
- [Pea07] Alison Pease. *A Computational Model of Lakatos-style Reasoning*. PhD thesis, University of Edinburgh, 2007.
- [PS20] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. arXiv:2009.03393, 2020.
- [ZT26] Jian Zhang and Si-Cheng Tan. Automated conjecturing and theorem finding: A survey. *Journal of Computer Science and Technology*, 41(1):46–66, 2026. DOI 10.1007/s11390-026-6040-0.